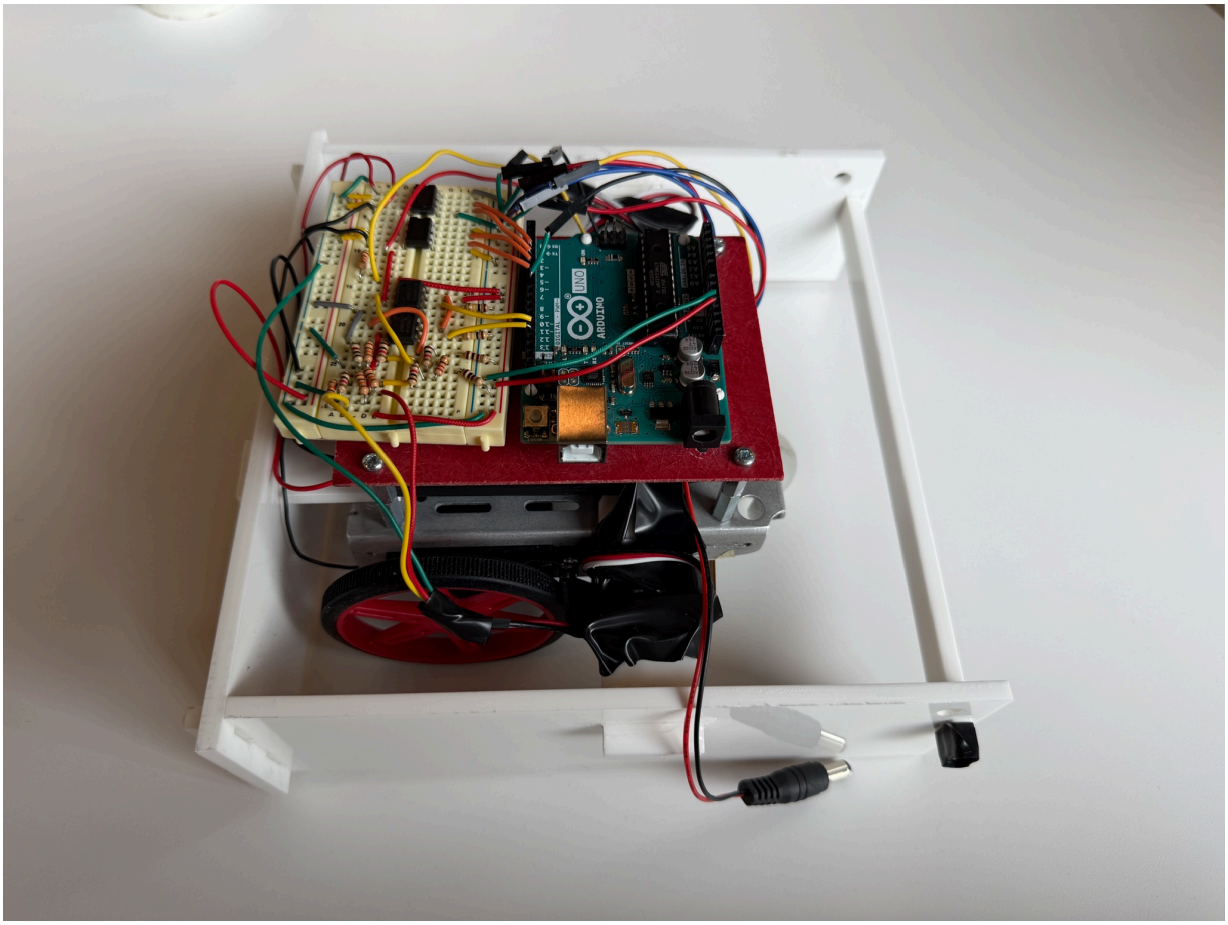
Mark Solis, mvs46
Sam Kuzmishin, shk94
Marcello Lombardo, ml2749
MAE 3780 Final Report, Group 30

Robot Design and Strategy Overview

Our robot's design revolved primarily around being able to stay within the boundaries of the board and collect cubes without losing them the entirety of the match. We discussed the competition with students in the class the prior year, and they revealed that given the limited computational power and simplistic code, simply being able to be on the board for the full minute allotted often led to victory, causing us to avoid complicated strategies in favor of using robust border detection. On the mechanical side, the team installed higher-quality wheels with more grip and designed a CAD model for an acrylic housing to collect cubes. Our logic was that by combining a robust chassis with this longevity on the board, we could collect and keep blocks, while potentially generating impacts, forcing other bots to lose them. The team had the idea to use sensors to precisely find cubes on the board, but with the software constraints of the competition, we were not able to effectively integrate them in a way that would improve our robot. Instead, we invested a lot of time in perfecting the QTI sensors and the color sensor to ensure consistent performance. We did this by creating a program that would repeat and use a black border detection to restart the algorithm.



Design Process Reflection

The team's robot went through many changes throughout the duration of the design process. The design we pitched for milestone 1 was very ambitious, although similar to our final results in some ways. While the housing for cube collection and overall structure stayed faithful to our milestone 1 design, our final strategy was very different . For the second milestone, the goal was just to get the robot to move. Using the H-bridges, batteries, the provided base, and the provided wheels, Marcello was able to get the robot working great. While working on milestone 3, our robot's flaws became apparent. For example, the plastic wheels with the added rubber bands for grip would disassemble very easily during testing. This was an obvious fault because if the wheels were to disassemble during the competition, our mobility would be greatly compromised due to the loss of grip and the possibility of the rubber bands to become caught around the axle of the motor and wheel. This heavily influenced our decision to purchase better wheels. We also had to update the structure of the robot to attach our color and QTI sensors. Using standoffs and rearranging the structure, Sam was able to attach the color sensor and the 2 QTI sensors on the bottom of the robot at the optimal distance for best performance. Marcello closed out the milestone with the wiring and code which only utilized the color sensor. The reason for only utilizing the color sensor was because when the QTIs were plugged directly into the Arduino, they consistently misfired and caused the robot to turn to avoid a barrier unnecessarily. This inconsistency caused it to never reach the center of the board and to sometimes go out of bounds. To remain competitive, the team knew the QTIs had to be implemented into the game plan. Since the QTIs gave different voltages depending on the shade of grey the color gave the sensor, it was clear that a comparator operational-amplifier had to be used to signal to the Arduino if a threshold voltage was passed (black gave the highest voltage). Mark handled the circuitry and testing for finding the proper threshold voltage to compare the QTI output to. If the threshold voltage was too high, there was a chance that the robot would ignore the boundaries and if the threshold was too low, the robot would misfire whenever it detected the color blue. A voltage divider was used to output the threshold voltage and by using different combinations of resistors, a consistent threshold voltage was found. (**Figure B.1** & **Figure B.2**). While this added complexity, it ultimately led to an ultra reliable border direction method, and was very useful in the Duffield lighting conditions. The final milestone, cube collection, required a major update to the structure of the robot. Marcello, using Solidworks and the RPL, created a CAD model (**Figure C.1**) for the housing of the robot, cube collection, and added locations to better place our sensors underneath the robot, all while meeting the constraints of dimensions and budget. The original design featured a one-way gate at the front of the robot to prevent any cubes exiting our housing whenever the robot reversed. In practice, the one-way gate did not always swing back down after letting a cube into the housing. To fix this, we introduced a cardboard "swinging door" to make the swinging easier.. This was enough to complete the final milestone but the team knew the structure needed to be refined to be competitive. The team's biggest learning point came from a "scrimmage" we had with another strong robot. Something we noted was that the one-way gate would jam because of cubes, which prevented them from entering the structure. The jamming of cubes also affected the wheels because it would sometimes prevent them from spinning freely. If the wheels could not spin freely it could prevent the robot from turning away from a border and generally make the robot less predictable. The team tried many different solutions, Sam added wheel guards with cardboard and added guards at the front wheel so the

cubes wouldn't get stuck in the crevices to help the one-way gate. We also tried different heights for the one-way gate to prevent it from getting caught on a cube. The wheels didn't jam as much with the new added guards but once enough cubes entered the structure it would eventually jam. The solution for the eventual wheel jamming came from Marcello's code. A side to side "jiggle" was implemented into the cycle to unlodge any cubes caught between the structure and wheel. This proved very effective for keeping the robot operating ideally. The final decision we made was to scrap the one-way gate, as we determined through testing that it was better to risk losing some cubes by reversing than not being able to collect cubes at all. Our opening plan would be to move diagonally to the center line and once the color change was detected, the robot would turn 90 towards the center of the board and cut back across the center line and from there the robot would stay within the borders and sweep around the board until the end of the match.

Competition Analysis

Overall, our robot performed extremely well during the competition. In the round robin portion, we went undefeated, which we feel is a credit to our strong cube enclosure and extremely reliable QTI logic. We went on to win one match in the final bracket before ultimately being eliminated by the second-place team in the round of eight. Everything on our robot worked as intended; what we feel ultimately cost us was our inability to smartly gather blocks via a search algorithm, or actively gather them with a physical mechanism. Overall, our simplistic design was our biggest strength and weakness, as its consistency allowed us to perform relatively well every match, but didn't necessarily take up an aggressive enough role to win in a single elimination tournament. Despite this, our design philosophy still produced results we are proud of and created a robot we still feel was capable of competing to win the tournament.

Conclusion

We feel the design choices we made produced a very capable robot. If we could implement a quick update with the benefit of hindsight, it would be to implement a switch on the breadboard that connected to the Arduino which would allow us to switch between 2 versions of the same strategy. If the switch was in the "off" position (no voltage to the input pin) it would be the program we ran on the day of the competition. If the switch was in the "on" position (5 V to the input pin) it would run the mirror version of the same program, where instead of having to position the robot to the left, we could position it to the right and follow our preferred path without collision. What ultimately led to our loss was colliding with the opponent's stronger robot. We started the robot in a specific orientation every single round, eventually someone was going to take note. We feel our budget allocation was adequate to complete our goals, although we would avoid purchasing additional sensors if we were to do this again. With hindsight, we would consider adding smart search capabilities, as well as potentially a more efficient gathering method. Our team's advice for students next year is to implement physical logic wherever possible to eliminate the possibility of software confusion, as well as to make sure every part of your robot is aligned for the same mission.

Appendix A: BOM

Bill Of Materials						
Part	**Fabrication Type***	**Bounding Box Perimeter****	**Mass*****	**Unit Cost**	**Quantity**	**Total**
Time of Flight Sensor	x	x	x	$14.95	1	**$14.95**
Wheels****	x	x	x	$2.00	2	**$4.00**
Acrylic Sheet	x	x	x	$5.00	1	**$5.00**
Back Plate	Laser Cut	21.000 in	x	$1.55	1	**$1.55**
Back Plate Mount	Laser Cut	8.260 in	x	$0.91	1	**$0.91**
Side Plate Mount	Laser Cut	5.880 in	x	$0.79	2	**$1.59**
Side Plate	Laser Cut	21.000 in	x	$1.55	2	**$3.10**
Gate	Laser Cut	17.500 in	x	$1.38	1	**$1.38**
Color Sensor Mount	Laser Cut	7.720 in	x	$0.89	1	**$0.89**
Gate Stop	Laser Cut	17.000 in	x	$1.35	1	**$1.35**
Gate Mount	3D Printed (PLA)	x	1.180 g	$1.47	2	**$2.94**
Total						**$37.66**

Figure A.1: Bill of Materials

* Per the 2026 RPL Guide, each acrylic sheet used (12 in × 12 in) was priced at $5.00. The cost of each laser-cut part was calculated using $0.05/in cut plus $0.50/part. Each PLA part was priced at $1.00/part plus $0.40/g.

** The bounding box for each laser-cut part was found using the SOLIDWORKS Bounding Box feature. The bounding box perimeter was used to calculate the inches cut per part, as specified in the 2026 RPL Guide. The bounding box dimensions for each part can be found in Appendix C.

*** The mass for each 3D-printed part was found using the SOLIDWORKS Mass Properties feature. The density used to calculate the mass of PLA parts was 1.25 g/cm³. This density was multiplied by 0.75 following the initial estimate guidance in the 2026 RPL Guide, which states: "... it is good to assume most parts will be 75% of the above density."

**** Following the guidelines from Ed Discussion Post #219, the ordered wheels were priced at $2.00 per wheel since the lab had run out of additional wheels.

Appendix B: Circuitry

The following are the comparator circuits. During testing, we noticed that the left QTI outputted a higher voltage for blue than the right QTI. A higher inverting input had to be used.

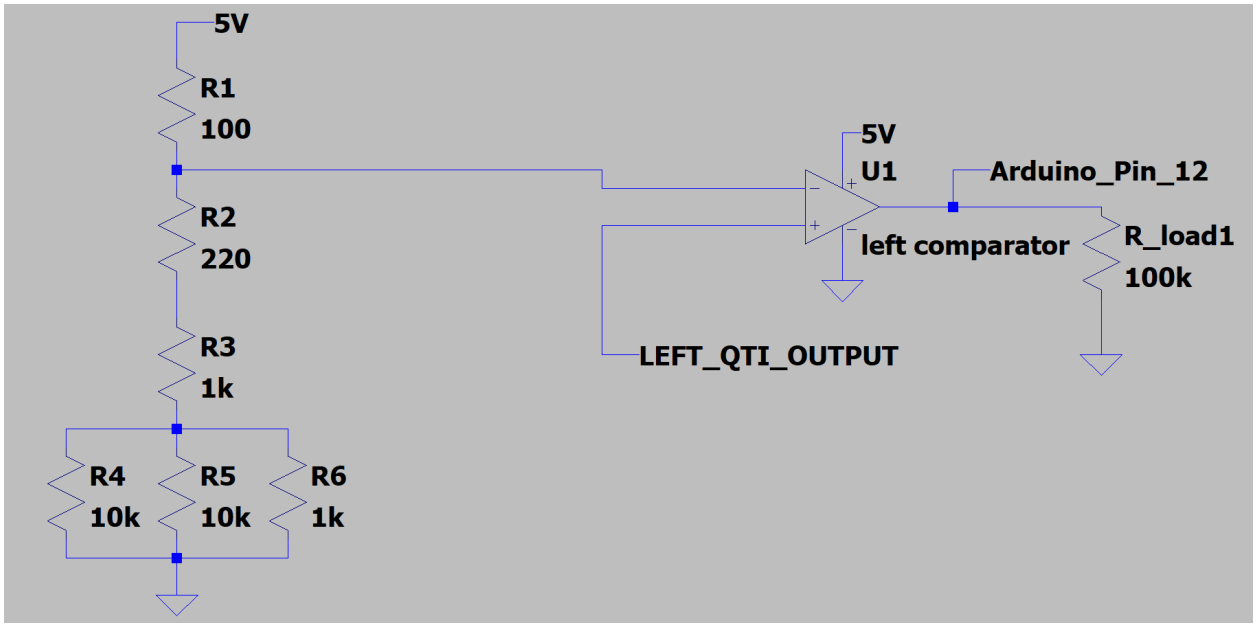


Figure B.1: The circuit schematic for the left QTI sensor - Inverting input is 4.768 V

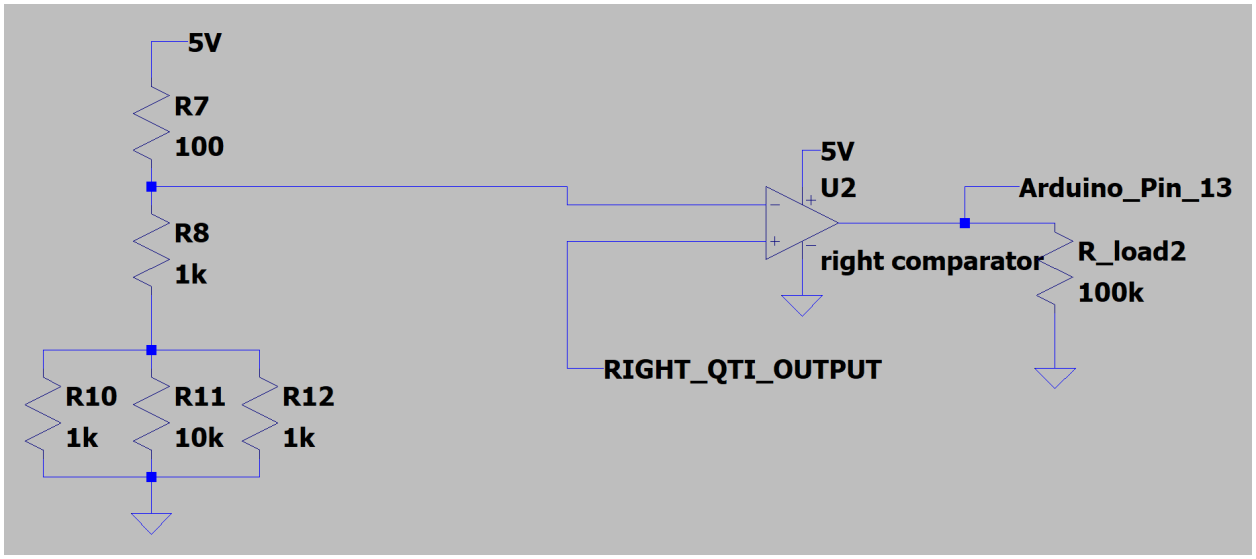


Figure B.2: The circuit schematic for the right QTI sensor - The inverting input is 4.683 V

Appendix C: CAD

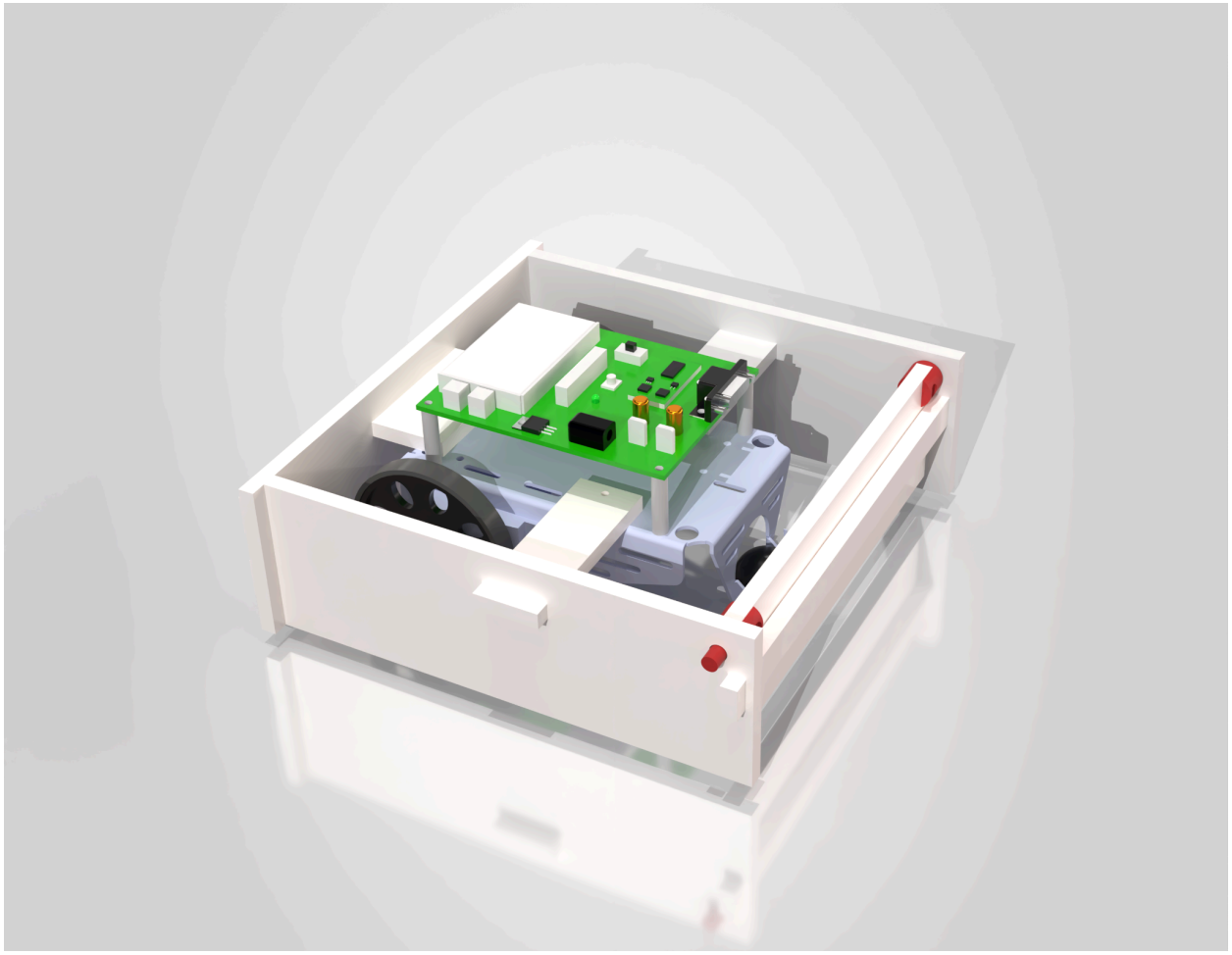


Figure C.1: CAD model for robot assembly

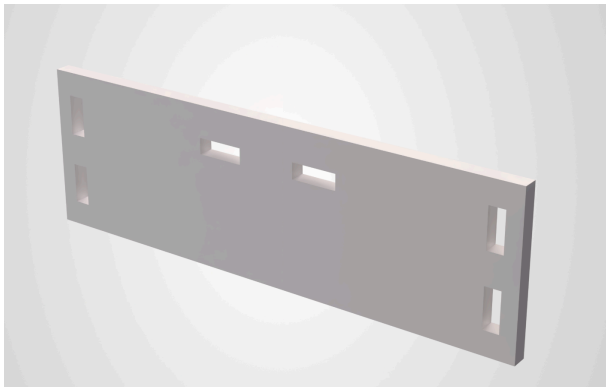


Figure C.2.A: Back Plate

Bounding Box

Total Bounding Box Length: 8in
Total Bounding Box Width: 2.5in
Total Bounding Box Thickness: 0.25in
Total Bounding Box Volume: 5 in^3

Figure C.2.B: Back Plate Bounding Box

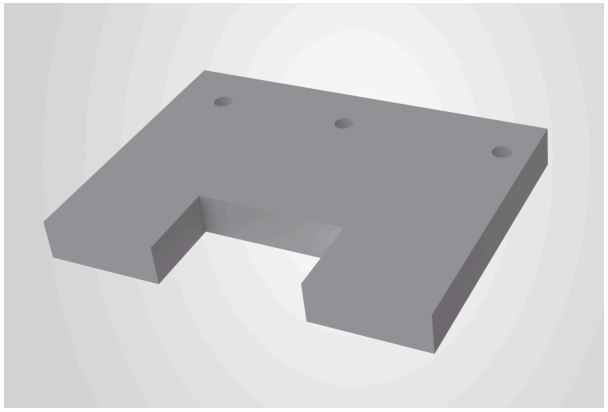


Figure C.3.A: Back Plate Mount

Bounding Box

Total Bounding Box Length: 2.36in
Total Bounding Box Width: 1.75in
Total Bounding Box Thickness: 0.25in
Total Bounding Box Volume: 1.03 in^3

Figure C.3.B: Back Plate Mount Bounding Box

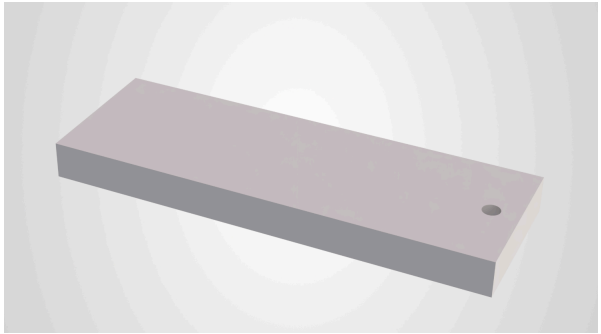


Figure C.4.A: Side Plate Mount

Bounding Box

Total Bounding Box Length: 2.94in
Total Bounding Box Width: 1in
Total Bounding Box Thickness: 0.25in
Total Bounding Box Volume: 0.74 in^3

Figure C.4.B: Side Plate Mount Bounding Box

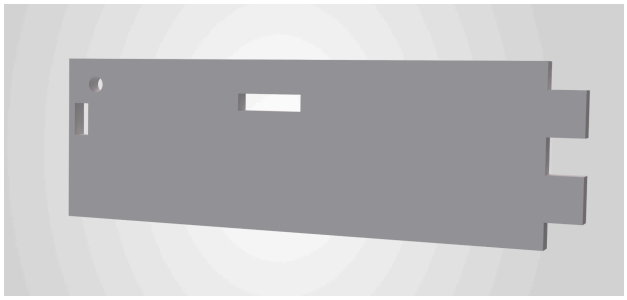


Figure C.5.A: Side Plate

Bounding Box

Total Bounding Box Length: 8in
Total Bounding Box Width: 2.5in
Total Bounding Box Thickness: 0.25in
Total Bounding Box Volume: 5 in^3

Figure C.5.B: Side Plate Bounding Box

Figure C.6.A: Gate

Bounding Box

Total Bounding Box Length: 6.75in
Total Bounding Box Width: 2in
Total Bounding Box Thickness: 0.25in
Total Bounding Box Volume: 3.37 in^3

Figure C.6.B: Gate Bounding Box

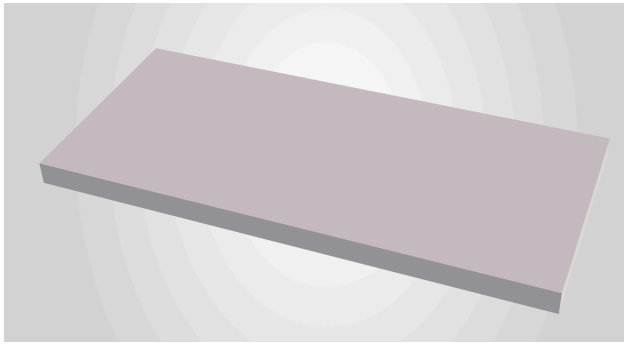


Figure C.7.A: Color Sensor Mount

Bounding Box

Total Bounding Box Length: 2.66in
Total Bounding Box Width: 1.2in
Total Bounding Box Thickness: 0.13in
Total Bounding Box Volume: 0.4 in^3

Figure C.7.B: Color Sensor Mount Bounding Box

Figure C.8.A: Gate Stop

Bounding Box

Total Bounding Box Length: 8in
Total Bounding Box Width: 0.5in
Total Bounding Box Thickness: 0.25in
Total Bounding Box Volume: 1 in^3

Figure C.8.B: Gate Stop Bounding Box

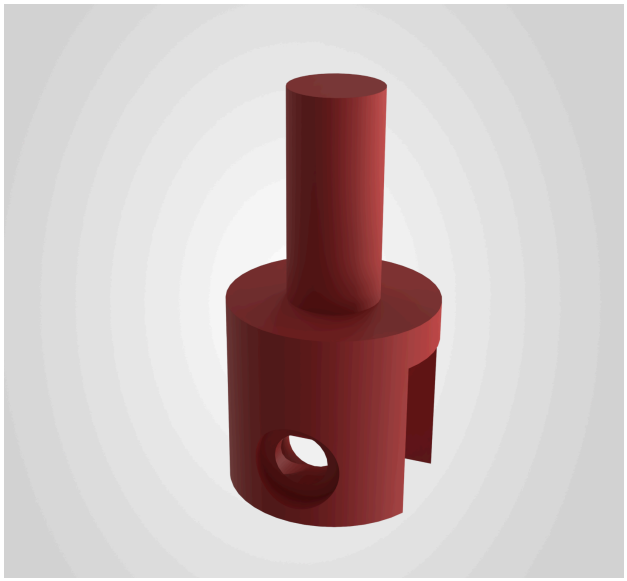


Figure C.9.A: Gate Mount

Mass properties of Gate Mount
Configuration: Default
Coordinate system: -- default --

Density = 0.94 grams per cubic centimeter

Mass = 1.18 grams

Volume = 1.25 cubic centimeters

Figure C.9.B: Gate Mount Mass Properties

Appendix D: Flowchart

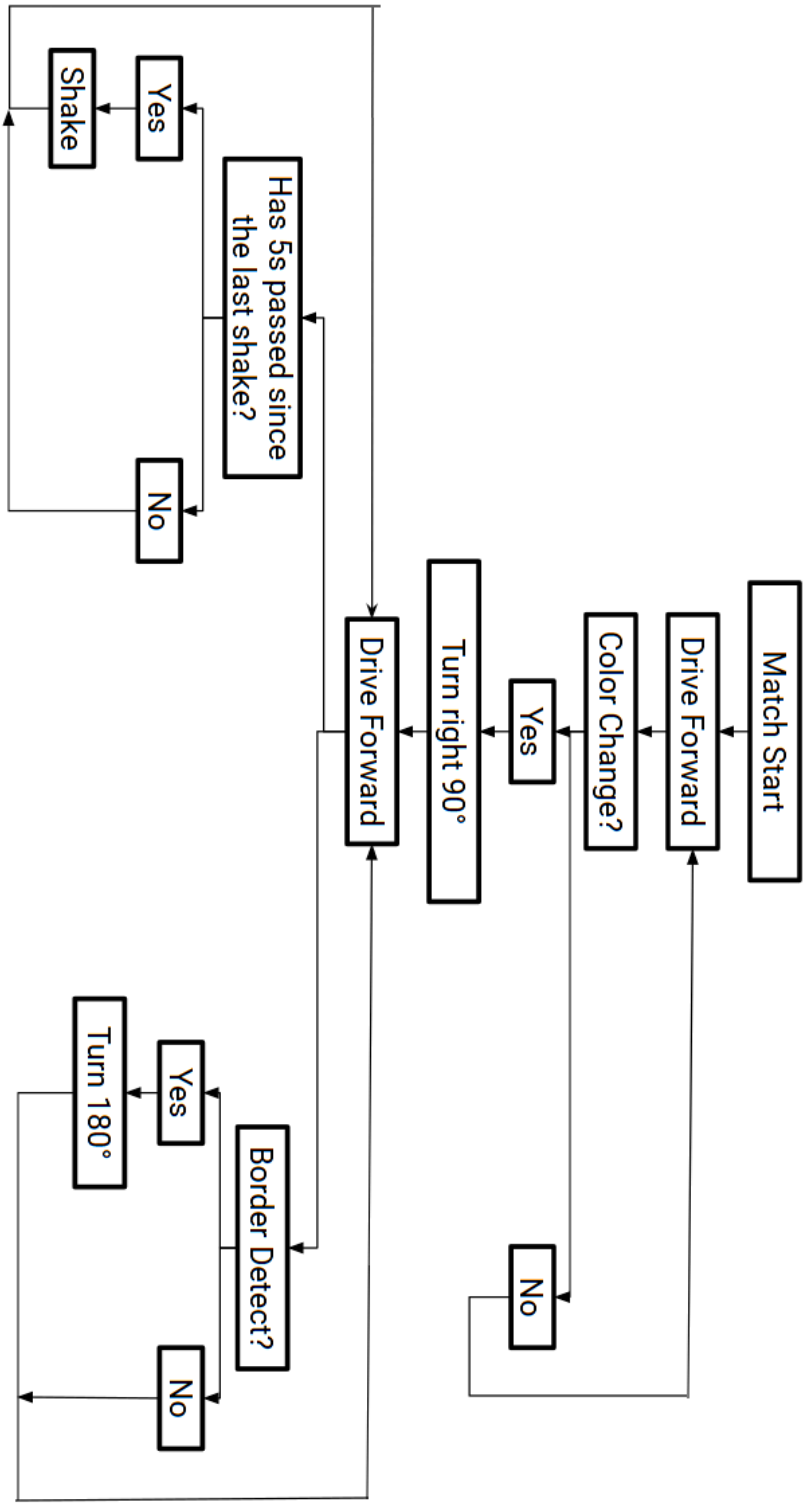


Figure D.1: Flowchart for robot's algorithm

Appendix E: Code

```
// ---------------------------
// ---------- ENUMS ----------
// ---------------------------

typedef enum {YELLOW, BLUE} Color;

typedef enum {
    DRIVE_UNTIL_NEW_COLOR,
    BACK_UP_AFTER_COLOR,
    TURN_RIGHT_AFTER_COLOR,
    DRIVE_FORWARD_12_IN,
    TURN_180_AFTER_12_IN,
    DRIVE_FORWARD_FOREVER
} RobotPhase;

// -------------------------------------
// ---------- GLOBAL VARIABLES ----------
// -------------------------------------

// Motors
uint8_t motor1_A = 0b00001000; // PD3
uint8_t motor1_B = 0b00010000; // PD4
uint8_t motor2_A = 0b01000000; // PD6
uint8_t motor2_B = 0b00100000; // PD5

uint16_t inch_time = 160; // ms per inch
uint16_t degree_time = 7; // ms per degree

// Color sensor
uint8_t color_pin = 0b10000000; // PD7

uint16_t period_threshold = 30; // us threshold to distinguish yellow vs
blue

// QTI border sensors
uint8_t qti_right_pin = 0b00010000; // PB4
uint8_t qti_left_pin  = 0b00100000; // PB5
```

```
volatile uint8_t border_right = 0; // right border detected
volatile uint8_t border_left = 0; // left border detected

uint16_t border_lockout_ms = 500; // ms to ignore new borders after
handling one
uint16_t last_border_time = 0; // last time a border was handled
uint8_t handling_border = 0; // flag to indicate currently handling a
border

// Jiggle timing
uint16_t jiggle_interval_ms = 10000; // jiggle every 10 seconds
uint16_t last_jiggle_time = 0; // last time the robot jiggle was performed

// Timers
volatile uint16_t timer0_ms = 0; // milliseconds since start
volatile uint16_t timer1_ticks = 0; // ticks for color sensor period
measurement

// --------------------------------
// ---------- TIMER SETUP ----------
// --------------------------------

void timer0_init(void) {
    TCCR0A = 0b00000010; // CTC mode
    TCCR0B = 0b00000011; // prescaler 64
    OCR0A = 249;         // 1 ms interrupt
    TCNT0 = 0;           // reset timer
    TIMSK0 = 0b00000010; // enable compare match A interrupt
}

void timer1_init(void) {
    TCCR1A = 0b00000000; // normal mode
    TCCR1B = 0b00000001; // prescaler 1
    TCNT1 = 0;           // reset timer
}

// --------------------------------
// ---------- INTERRUPTS ----------
// --------------------------------
```

```
// Timer0 compare match A interrupt - called every 1 ms
ISR(TIMER0_COMPA_vect) {
    timer0_ms++; // increment millisecond counter
}

// Color sensor rising and falling edge interrupt
ISR(PCINT2_vect) {
    if (PIND & color_pin) {
        TCNT1 = 0; // reset timer on rising edge
    }
    else {
        timer1_ticks = TCNT1; // capture timer ticks on falling edge
    }
}

// QTI border sensors interrupt
ISR(PCINT0_vect) {
    if (PINB & qti_right_pin) {
        border_right = 1; // right border detected
    }
    else {
        border_right = 0; // right border cleared
    }

    if (PINB & qti_left_pin) {
        border_left = 1; // left border detected
    }
    else {
        border_left = 0; // left border cleared
    }
}

// --------------------------------
// ---------- WAIT LOGIC ----------
// --------------------------------

// non blocking delay function that allows interrupts to run
void wait_ms(uint16_t ms) {
    uint16_t start_time = timer0_ms; // capture start time
```

```
    while ((uint16_t)(timer0_ms - start_time) < ms) {
        // wait
    }
}

// ------------------------------------------------
// ---------- BORDER HANDLING FUNCTIONS ----------
// ------------------------------------------------

// check if enough time has passed since last border to allow handling a
new one
uint8_t border_allowed(void) {
    if ((uint16_t)(timer0_ms - last_border_time) > border_lockout_ms) {
        return 1; // allowed to handle new border
    }
    else {
        return 0; // still in lockout period, ignore new borders
    }
}

// check if either border sensor is currently triggered
uint8_t border_detected(void) {
    if (border_right == 1 || border_left == 1) {
        return 1; // border detected
    }
    else {
        return 0; // no border detected
    }
}

// wait for specified ms while also checking for borders
uint8_t wait_ms_check_border(uint16_t ms) {
    uint16_t start_time = timer0_ms;

    while ((uint16_t)(timer0_ms - start_time) < ms) {
        if (handling_border == 0 && border_allowed() && border_detected())
{
            return 1; // border detected during wait
        }
    }
```

```
    return 0; // wait completed without border detection
}

// ---------------------------------
// ---------- SENSOR SETUP ----------
// ---------------------------------

// initialize color sensor pin and interrupt
void init_color_sensor(void) {
    DDRD &= ~color_pin;      // D7 input
    PCICR |= 0b00000100;     // enable pin-change interrupt for Port D
}

// initialize QTI sensor pins and interrupt
void init_qti_sensors(void) {
    DDRB &= ~(qti_right_pin | qti_left_pin);  // D12 & D13 input

    PCICR |= 0b00000001;                      // enable pin-change
interrupt for Port B
    PCMSK0 |= (qti_right_pin | qti_left_pin); // enable pin-change
interrupt for D12 & D13
}

// ---------------------------------------
// ---------- COLOR SENSOR LOGIC ----------
// ---------------------------------------

// measure the period of the color sensor signal in microseconds
uint16_t get_period(void) {
    timer1_ticks = 0;

    PCMSK2 |= color_pin;
    wait_ms(5);
    PCMSK2 &= ~color_pin;

    uint16_t period = 2 * timer1_ticks / 16;
    return period;
}
```

```
// determine color based on period measurement
Color get_color_from_period(uint16_t period) {
    if (period < period_threshold) {
        return YELLOW;
    }
    else {
        return BLUE;
    }
}

// -----------------------------------
// ---------- MOTOR CONTROL ----------
// -----------------------------------

// initialize motor control pins
void init_motors(void) {
    DDRD |= (motor1_A | motor1_B | motor2_A | motor2_B);
    PORTD &= ~(motor1_A | motor1_B | motor2_A | motor2_B);
}

// stop all motors
void stop(void) {
    PORTD &= ~(motor1_A | motor1_B | motor2_A | motor2_B);
}

// drive forward indefinitely
void drive_forward_forever(void) {
    PORTD &= ~(motor1_A | motor1_B | motor2_A | motor2_B);
    PORTD |= (motor1_A | motor2_A);
}

// drive foward for specified inches, return 1 if border detected during
drive
uint8_t drive_forward(uint16_t inches) {
    PORTD &= ~(motor1_A | motor1_B | motor2_A | motor2_B);
    PORTD |= (motor1_A | motor2_A);

    if (inches == 0) {
        return 0;
    }
```

```
    if (wait_ms_check_border(inch_time * inches)) {
        stop();
        return 1;
    }

    stop();
    return 0;
}

// drive backward for specified inches, return 1 if border detected during
drive
uint8_t drive_backward(uint16_t inches) {
    PORTD &= ~(motor1_A | motor1_B | motor2_A | motor2_B);
    PORTD |= (motor1_B | motor2_B);

    if (inches == 0) {
        return 0;
    }

    if (wait_ms_check_border(inch_time * inches)) {
        stop();
        return 1;
    }

    stop();
    return 0;
}

// turn right for specified degrees, return 1 if border detected during
turn
uint8_t turn_right(uint16_t degrees) {
    PORTD &= ~(motor1_A | motor1_B | motor2_A | motor2_B);
    PORTD |= (motor1_A | motor2_B);

    if (wait_ms_check_border(degree_time * degrees)) {
        stop();
        return 1;
    }
```

```
    stop();
    return 0;
}

// turn left for specified degrees, return 1 if border detected during
turn
uint8_t turn_left(uint16_t degrees) {
    PORTD &= ~(motor1_A | motor1_B | motor2_A | motor2_B);
    PORTD |= (motor1_B | motor2_A);

    if (wait_ms_check_border(degree_time * degrees)) {
        stop();
        return 1;
    }

    stop();
    return 0;
}

// ---------------------------------
// ---------- BORDER LOGIC ----------
// ---------------------------------

// handle border detection and response
void handle_border(void) {
    if (handling_border == 1) {
        return; // already handling a border, ignore new detections
    }

    if (!border_allowed()) {
        return; // still in lockout period, ignore new borders
    }

    if (border_right == 0 && border_left == 0) {
        return; // no border currently detected
    }

    handling_border = 1; // set flag to indicate handling a border
    last_border_time = timer0_ms; // capture time of border detection
```

```
    stop(); // stop immediately when border detected
    wait_ms(100); // stop for a moment

    // both borders triggered
    if (border_right == 1 && border_left == 1) {
        Serial.print("BORDER: BOTH\r\n"); //debugging serial print

        drive_backward(2); // back up 2 inches
        wait_ms(100); // wait 100 ms

        turn_right(180); // turn 180 degrees
        wait_ms(100); // wait 100 ms
    }
    // right border triggered
    else if (border_right == 1) {
        Serial.print("BORDER: RIGHT\r\n"); //debugging serial print

        drive_backward(1); // back up 1 inch
        wait_ms(100); // wait 100 ms

        turn_left(120); // turn left 120 degrees
        wait_ms(100); // wait 100 ms
    }
    // left border triggered
    else if (border_left == 1) {
        Serial.print("BORDER: LEFT\r\n"); //debugging serial print

        drive_backward(1); // back up 1 inch
        wait_ms(100); // wait 100 ms

        turn_right(120); // turn right 120 degrees
        wait_ms(100); // wait 100 ms
    }

    border_right = 0; // reset border flags after handling
    border_left = 0; // reset border flags after handling

    last_border_time = timer0_ms; // reset lockout timer
    handling_border = 0; // reset handling flag
```

```
    drive_forward_forever();
}


//---------------------------------
// ---------- JIGGLE LOGIC ---------
// ---------------------------------

// perform a jiggle maneuver to try to free the robot if it's stuck
void jiggle_robot(void) {
    uint16_t jiggle_start_time = timer0_ms; // capture start time of
jiggle

    Serial.print("JIGGLE\r\n"); // debugging serial print

    // alternate between turning right and left
    while ((uint16_t)(timer0_ms - jiggle_start_time) < 2000) {
        handle_border(); // check for borders before each turn

        // turn right for 10 degrees
        if (turn_right(10)) {
            handle_border();
            return;
        }

        wait_ms(50); // wait 50 ms

        handle_border();

        // turn left for 10 degrees
        if (turn_left(10)) {
            handle_border();
            return;
        }

        // wait 50 ms
        wait_ms(50);
    }

    // after jiggle maneuver, resume driving forward
```

```
    drive_forward_forever();
}

// check if it's time to perform a jiggle and do it if needed
void jiggle_if_needed(void) {
    if ((uint16_t)(timer0_ms - last_jiggle_time) >= jiggle_interval_ms) {
        last_jiggle_time = timer0_ms;
        jiggle_robot();
    }
}

// -------------------------------------
// ---------- SERIAL PRINTING ----------
// -------------------------------------

void print_status(uint16_t period, Color color) {
    Serial.print("Period: ");
    Serial.print(period);

    Serial.print(" us  Color: ");
    if (color == YELLOW) {
        Serial.print("YELLOW");
    }
    else {
        Serial.print("BLUE");
    }

    Serial.print("  Border Right: ");
    Serial.print(border_right);

    Serial.print("  Border Left: ");
    Serial.print(border_left);

    Serial.print("\r\n");
}


// -------------------------
// ---------- MAIN ----------
// -------------------------
```

```
int main(void) {
    // --- initialization ---
    timer0_init();
    timer1_init();
    init_color_sensor();
    init_motors();
    init_qti_sensors();

    sei();              // master interrupts ON
    Serial.begin(9600); // allowed for debugging

    // --- variables ---
    Color start_color;
    Color current_color;
    uint16_t current_period;

    RobotPhase robot_phase = DRIVE_UNTIL_NEW_COLOR;

    current_period = get_period();
    start_color = get_color_from_period(current_period);

    last_jiggle_time = timer0_ms;

    drive_forward_forever();

    while (1) {
        current_period = get_period();
        current_color = get_color_from_period(current_period);

        print_status(current_period, current_color);

        handle_border();

        switch (robot_phase) {
            case DRIVE_UNTIL_NEW_COLOR:
                if (current_color != start_color) {
                    stop();
                    wait_ms(100);
```

```
                robot_phase = BACK_UP_AFTER_COLOR;
            }
            else {
                drive_forward_forever();
                jiggle_if_needed();
            }
            break;

        case BACK_UP_AFTER_COLOR:
            if (drive_backward(3)) {
                handle_border();
            }
            else {
                wait_ms(100);
                robot_phase = TURN_RIGHT_AFTER_COLOR;
            }
            break;

        case TURN_RIGHT_AFTER_COLOR:
            if (turn_right(75)) {
                handle_border();
            }
            else {
                wait_ms(100);
                robot_phase = DRIVE_FORWARD_12_IN;
            }
            break;

        case DRIVE_FORWARD_12_IN:
            if (drive_forward(12)) {
                handle_border();
            }
            else {
                wait_ms(100);
                robot_phase = TURN_180_AFTER_12_IN;
            }
            break;

        case TURN_180_AFTER_12_IN:
            if (turn_right(180)) {
```

```
                    handle_border();
                }
                else {
                    wait_ms(100);
                    robot_phase = DRIVE_FORWARD_FOREVER;
                }
                break;

            case DRIVE_FORWARD_FOREVER:
                drive_forward_forever();
                jiggle_if_needed();
                break;
        }

        wait_ms(50);
    }
}
```